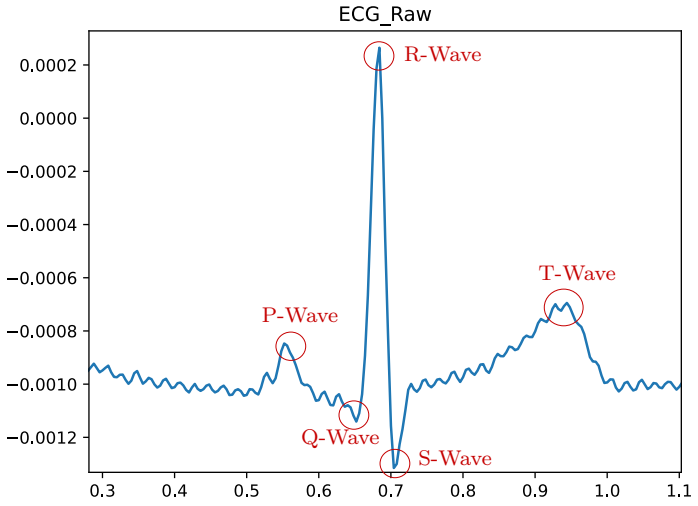
Digital Signal Processing
ECG FIR Filtering Assignment

2785209M & 2664680S

1 Plotting the ECG Signal

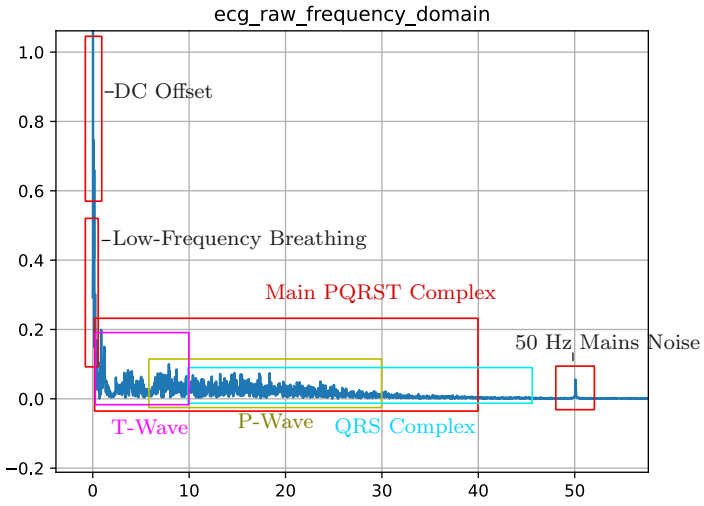
The experiment began by taking an ECG recording and plotting it in python using matplotlib. The electrodes were attached just above the hip and on the front of the shoulder.

1.1 Structure of PQRST Complex



It is important that we preserve the frequencies used to construct the PQRST complex, otherwise we would be destroying important data from the ECG.

1.2 Sources of Noise in the ECG



The main sources of noise in an ECG are 50Hz mains interference, Breathing, and random noise caused by movement. There is also a DC offset which shifts the entire signal by a certain amount, caused by potential difference formed between the electrodes and the skin.
The human heart rate can go as low as 0.5Hz at complete rest. Considering this, the filter should remove frequencies below 0.5Hz (This includes DC shift and Breathing) and above 45Hz (Mains noise and other high frequency interference). Although this leaves in random noise cause by movement, the core frequencies of the PQRST complex must be retained for the signal to remain accurate for monitoring purposes.

2 Generating the FIR Coefficients

2.1 Stop-Band Filter

A stop-band filter was implemented in this code by utilizing an all-pass filter, a band-pass filter, and the principle of superposition. This was done instead of using a pure band-stop filter in order to keep the code simple and elegant, as it is easier to remove frequencies from an all-pass filter than it is to combine multiple separate lo-pass and hi-pass filters together.
First, the number of coefficients needed for the windowing function to work had to be calculated. For a Kaiser Window, this changes based on the desired stopband attenuation and transition width. The formula is displayed in the next subsection in Equation (3). Then, using the number of taps, an ideal all-pass filter was defined by creating an array of zeros, with an index stretching from indices of -m to m (where m = num_taps / 2), with the central value at index m set to 1. This represents the ideal impulse response. Finally the array n is initialized, which represents discrete time indices from -m to m. Evalutating functions over this symmetric time vector is what gives the filter linear phase and constant group delay.
When a signal is convolved with a unit impulse, it is left unchanged; this is what makes it all-pass. This all-pass filter is then modified in a for loop iterating over the array of stop bands. The stop bands live in the frequency domain, so an inverse discrete time fourier transform is applied to convert them into the time-domain before they are subtracted from the impulse response:

$$h_{\text{band}}[n] = 2 \cdot f_2 \cdot \text{sinc}(2 \cdot f_2 \cdot n) - 2 \cdot f_1 \cdot \text{sinc}(2 \cdot f_1 \cdot n) \tag{1}$$

(Note: Doing the IDFT manually is less efficient than using np.IFFT() and was done purely for analytical purposes)

After this for-loop is complete, h will be a raw band-stop filter (sharp edges in the frequency domain, causing ringing due to the Gibbs Phenomenon). To smooth out these edges, it must be multiplied with a windowing function.

2.2 The Kaiser Window

The Kaiser window was chosen for its flexibility and simplicity. It gives greater flexibility than other windowing functions, since the user has control over both the size of the transition window and the stopband attenuation. This allows you to find a precise balance between a narrow central peak (sharper cutoff) and flatter sidelobes (better noise rejection) in the impulse response. With specific stopband attenuations, it is also possible for the Kaiser window to mimic other windowing functions. Furthermore, the Kaiser filter has straightforward formulas to calculate beta (the stopband attenuation parameter) and num_taps (the number of coefficients needed for the filter), allowing it to be practically calculated almost entirely in code without external libraries. The formulae used in the code are as follows:

beta parameter:

$$\beta = \begin{cases} 0.1102(A - 8.7) & \text{if } A > 50 \\ 0.5842(A - 21)^{0.4} + 0.07886(A - 21) & \text{if } 21 \leq A \leq 50 \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

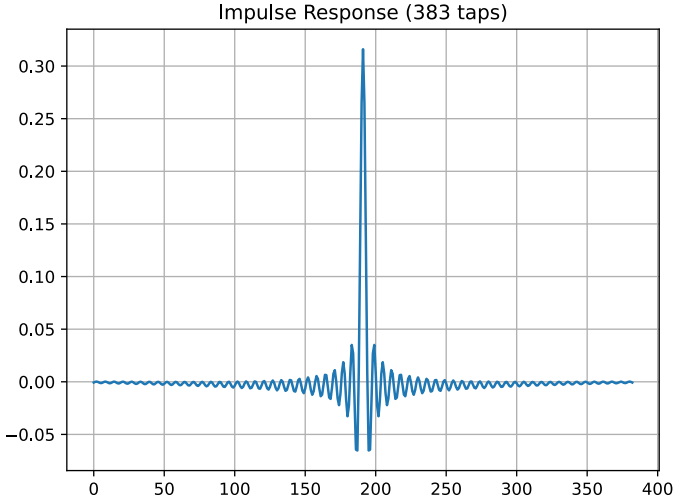
number of taps:

$$N = \left\lfloor \frac{A - 8}{2.285 \cdot 2\pi \left(\frac{\Delta f}{f_s}\right)} \right\rfloor \tag{3}$$

kaiser window:

$$w[n] = \frac{I_0\left(\beta\sqrt{1 - \left(\frac{n}{m}\right)^2}\right)}{I_0(\beta)} \tag{4}$$

The FIR filter is then multiplied by the Kaiser window, to create the final array of coefficients, resulting in this impulse response:

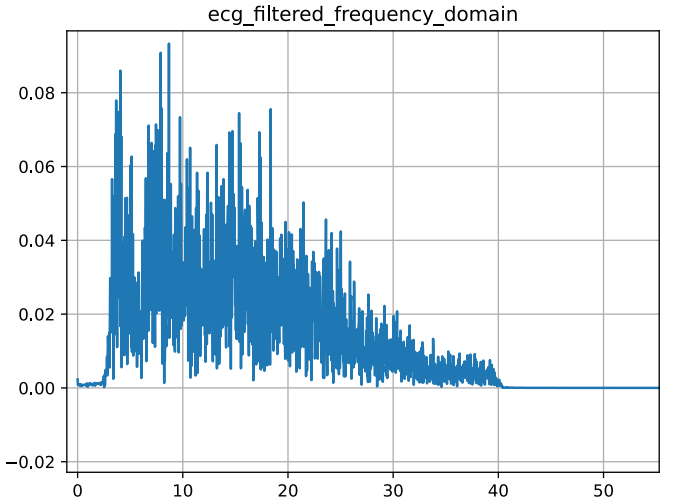


As you can see, this results in an impulse response with a large central peak and sidelobes that gradually decay to zero as they approach tap 0 and tap 383. This shows that the Kaiser Window has successfully truncated and reduced the sharpness of our FIR filter. These coefficients can now be passed into the fir_filter class to simulate realtime filtering.

3 Simulation of Real-time Filtering

3.1 Subsection

$$MathBlock \tag{5}$$



tiny text Normal Sized text large text **bold text**

4 Adaptive LMS Filter

4.1 Subsection

5 Heartbeat Detection

5.1 Subsection